Unit 8 - Comparing Machine Learning, Deep Learning and Ethical Impact in Practice – Seminar Tasks

Task 1 - Model Comparison in Practice

Yu et al. (2024) tackle a practical forecasting problem in supply chain management: predicting supply chain distribution cost (SCMDC), a figure made up of transportation, warehousing, handling, and delivery expenses. Because these costs shift constantly with demand, shipping mode, and delivery timing, traditional estimation methods struggle to keep up. The aim was to test whether deep learning could do better than standard machine learning at capturing these patterns, using a large open dataset of over 180,000 records covering clothing, sports, and electronics orders.

The comparison is grounded in hard numbers rather than general claims. Four conventional models (random forest, support vector machine, multilayer perceptron, and decision tree) were tested against a convolutional neural network on the same train/test/validation split, using RMSE, R^2, mean relative error, and error distribution plots. The CNN came out on top with an R^2 of 0.953 and RMSE of 0.528, beating decision tree, random forest, SVM, and MLP in that order. Error histograms and cross-plots reinforced this, showing the CNN's predictions clustered more tightly around actual values with a narrower error spread.

However, the paper is honest about trade-offs. The CNN needed more computation to train and offered little insight into which features mattered most, or why a specific prediction came out the way it did. The simpler models, especially decision trees, were easier to interpret and cheaper to run, even if less accurate. The conventional models also relied more on manual feature engineering, while the CNN learned patterns directly from the data with less preprocessing. This makes the CNN attractive for large-scale, high-stakes prediction tasks, but conventional models remain reasonable choices when transparency, speed, or limited computing resources take priority.

The same trade-off applies to recommender systems: a deep model may be a little more accuracy in predicting what a user wants next, but a simpler model is often preferred in production because it's faster to retrain, easier to debug, and easier to explain to a product team when recommendations go wrong.

Task 2 - Explainability and Trust in AI Systems

Reading this study from the perspective of someone who must answer for the model's output, not just build it, changes what ‘the CNN wins’ actually means.

In a recommender setting, models like these are making decisions that shape what a user sees first, what gets pushed to the top of a feed, or which product gets flagged as a good fit. Nobody thinks of these as high-stakes decisions the way a medical diagnosis is, but they still affect what people buy, watch, or believe is popular. If I were on a team shipping something like this, my worry wouldn't be whether the CNN is 3 points of R^2 better than the random forest. It would be whether anyone on the team, including me, could explain why a specific user got a specific recommendation.

The concern with an unexplainable model isn't abstract. If a user or a business stakeholder asks ‘why did the system recommend this instead of that,’ and the honest answer is ‘the weights say so,’ it erodes trust, makes debugging biased or broken recommendations much harder, and it makes it nearly impossible to catch cases where the model has quietly learned to lean on something it shouldn't, like reinforcing narrow content just because it drives more clicks. An RF or DT provides a rough map of which features are pulling weight. A CNN, especially one with several convolutional and dense layers like the one in this paper, gives you a black box with strong accuracy and very little to point to when someone asks for a reason.

This is why explainability methods such as SHAP and LIME are relevant. SHAP values can show, after the fact, how much each input feature pushed a specific prediction up or down, which turns a black box into something a product manager or support team can read. LIME is similar by approximating the model's behaviour locally around one prediction with a simpler, interpretable model. Visual explanations, like saliency maps or feature importance charts, help less technical stakeholders get a feel for what's driving outputs without needing to understand the underlying math.

For my own role on a development team, I'd treat explainability as part of the deliverable and not an afterthought following deployment. That means building in a SHAP or LIME layer alongside any deep

model from the start, documenting what the top features are for typical predictions, and being ready to walk a stakeholder through a specific decision in plain language. It also means being upfront when a model's accuracy gain isn't worth the loss in interpretability and pushing back on the assumption that better accuracy automatically means it’s a better choice for production. A model that's slightly less accurate but is explainable to a user or a manager is the safer bet, especially early on when trust in the system still has to be earned.